Generative Artificial Intelligence

CycleGAN Assignment

# Question 2

## CycleGAN Implementation for Person Face Sketches

Bidirectional Face Photo and Sketch Translation


| Member | Roll Number |
|---|---|
| Uzair Majeed | 23i-3063 |
| Faez Ahmed | 23i-0598 |

**Section:** SE-B

**Instructor:** Shahela Saif


Implementation and Experimental Report

October 2026

# Contents

**1 Notebook Pipeline and Project Overview** **2**

**2 Dataset and Preprocessing** **3**

2.1 Dataset organization . . . . . 3

2.2 Notebook input pipeline . . . . . 3

**3 CycleGAN Architecture** **4**

3.1 ResNet-6 generators . . . . . 4

3.2 PatchGAN discriminators . . . . . 5

**4 Losses and Training Procedure** **6**

4.1 Adversarial, cycle, and identity objectives . . . . . 6

4.2 Optimization and training configuration . . . . . 6

**5 Experimental Results** **7**

5.1 Evaluation protocol . . . . . 7

5.2 Loss measurements . . . . . 7

**6 Checkpointing and Model Export** **11**

**7 Flask Backend and React Interface** **12**

7.1 Application workflow . . . . . 12

7.2 Automatic input detection . . . . . 12

7.3 Application preprocessing and optional enhancement . . . . . 12

7.4 Sketch-to-Photo Translation Demo . . . . . 14

7.5 Photo-to-Sketch Translation Demo . . . . . 15

**8 Verification and Limitations** **16**

**9 Conclusion** **18**

**A Artifacts and Run Commands** **18**

# 1 Notebook Pipeline and Project Overview

The implementation starts in `cyclegan-q2-notebook.ipynb`. Its objective is to learn two mappings: a face photograph to a sketch and a sketch to a plausible face photograph. The trained generators are exported for a local Flask API and React interface. The approach follows the cycle-consistent, unpaired translation formulation of Zhu et al. [1].

The notebook pipeline proceeds through:

1. Dataset inspection and visualization of photo/sketch examples.
2. Separate TensorFlow input pipelines for photographs and sketches.
3. Construction of two ResNet generators and two PatchGAN discriminators.
4. Definition of adversarial, cycle-consistency, and identity losses.
5. Initialization of optimizers, replay buffers, and checkpoint restoration.
6. End-to-end training with loss logging, checkpoints, and visual samples.
7. Validation/test evaluation and export of generators and weights.
8. Integration of the exported generators into the Flask/React application.

**Reported run:** Saved history, evaluation JSON files, and notebook restoration output establish **11 completed epochs**. The last training call requests 12 epochs, but this target is not evidence that epoch 12 completed. Final metrics in this report refer to epoch 11.

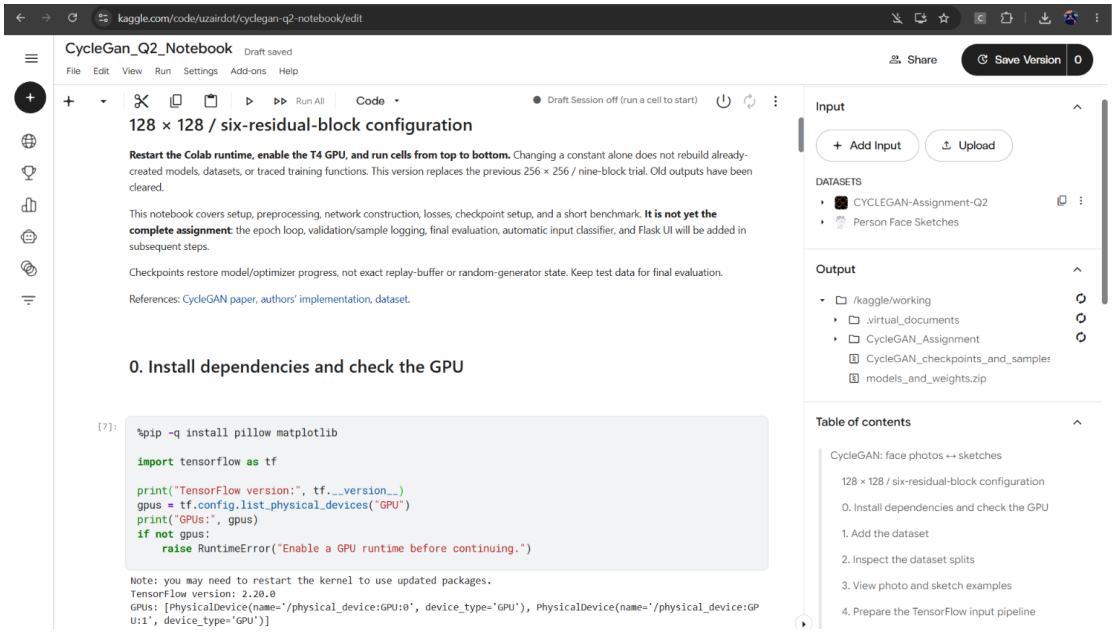


**Figure 1:** Dataset-to-notebook-to-application pipeline, including training, checkpointing, evaluation, and model export.

# 2 Dataset and Preprocessing

## 2.1 Dataset organization

The Person Face Sketches dataset is supplied through Kaggle [2]. The notebook records the counts in Table 1. Matching filenames support inspection, but training independently shuffles the two domains rather than using paired supervised targets.

**Table 1:** Dataset counts recorded in notebook outputs.

| Split | Photos | Sketches | Total files |
|---|---|---|---|
| Training | 20,655 | 20,655 | 41,310 |
| Validation | 1,000 | 1,000 | 2,000 |
| Test | 679 | 679 | 1,358 |
| Total | 22,334 | 22,334 | 44,668 |

Source images are described in the project brief as $512 \times 512$, with RGB photographs and grayscale sketches. The existing split is preserved. The full dataset is not included in the local application workspace; these counts come from saved notebook outputs.

## 2.2 Notebook input pipeline

Images are decoded as three-channel RGB, repeating grayscale values across channels when needed. The *whole image* is resized to $128 \times 128$ using TensorFlow bilinear interpolation with `antialias=True`. Float32 pixels are normalized by

$$x_{\text{norm}} = x/127.5 - 1. \tag{1}$$

No face crop or sketch-cleanup filter is applied in the notebook pipeline. Training uses random horizontal flips, independent domain shuffles with seeds 42 and 123, and batch size one. Validation/test inputs are not shuffled or augmented. Images are loaded on demand through `tf.data`, parallel mapping, and prefetching instead of loading the complete dataset into memory.

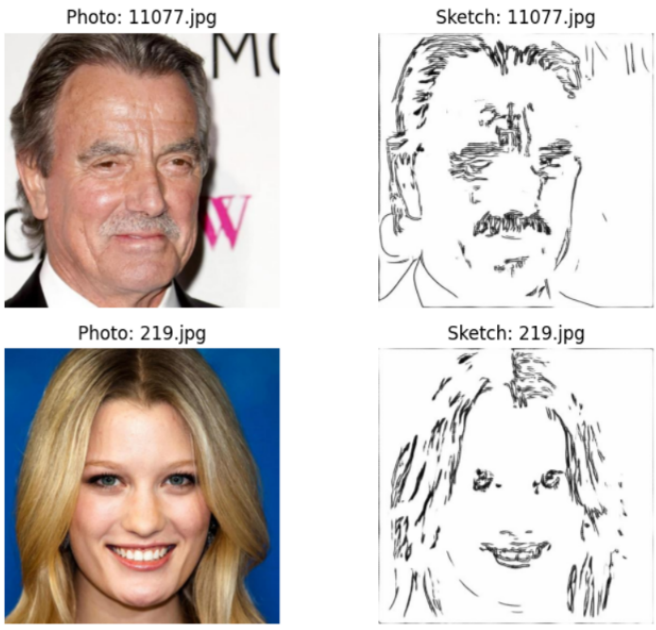


**Figure 2:** Original photographs and sketches illustrating the two training domains.

# 3 CycleGAN Architecture

Let $X$ be the photo domain and $Y$ the sketch domain. The networks are $G : X \to Y$, $F : Y \to X$, sketch discriminator $D_Y$, and photo discriminator $D_X$. Their joint training encourages target-domain appearance and reconstruction through the reverse mapping.

## 3.1 ResNet-6 generators

Each generator has **7,845,123 parameters**. Both use reflection padding, instance normalization, ReLU, and final tanh activation. Convolution kernels use normal initialization with mean zero and standard deviation 0.02. Instance normalization has trainable scale/offset and $\epsilon = 10^{-5}$.

**Table 2:** Generator architecture.

| Stage | Operation | Output shape |
|---|---|---|
| Input | Normalized RGB image | $128 \times 128 \times 3$ |
| Initial | Padded $7 \times 7$ convolution; 64 filters; normalization and ReLU | $128 \times 128 \times 64$ |
| Downsample 1 | $3 \times 3$ convolution; stride 2; 128 filters | $64 \times 64 \times 128$ |
| Downsample 2 | $3 \times 3$ convolution; stride 2; 256 filters | $32 \times 32 \times 256$ |
| Residual stack | Six blocks; two padded $3 \times 3$ convolutions and skip addition per block | $32 \times 32 \times 256$ |
| Upsample 1 | Transposed convolution; stride 2; 128 filters | $64 \times 64 \times 128$ |
| Upsample 2 | Transposed convolution; stride 2; 64 filters | $128 \times 128 \times 64$ |
| Output | Padded $7 \times 7$ convolution; 3 filters; tanh | $128 \times 128 \times 3$ |

### 3.2 PatchGAN discriminators

Each discriminator has **2,766,529 parameters**. Four stages use $4 \times 4$ convolutions with 64, 128, 256, and 512 filters and strides 2, 2, 2, and 1. The first stage omits normalization; LeakyReLU uses slope 0.2. A final convolution yields a $14 \times 14 \times 1$ patch-score grid with the implemented $70 \times 70$ receptive field. No sigmoid is applied because the adversarial objective uses least squares.

# 4 Losses and Training Procedure

## 4.1 Adversarial, cycle, and identity objectives

The photo-to-sketch adversarial loss is

$$\mathcal{L}_{\text{adv}}^{G} = \mathbb{E}_x[(D_Y(G(x)) - 1)^2], \tag{2}$$

with the analogous reverse loss using $D_X(F(y))$. The sketch-discriminator loss is

$$\mathcal{L}_{D_Y} = \tfrac{1}{2}\mathbb{E}_y[(D_Y(y) - 1)^2] + \tfrac{1}{2}\mathbb{E}_x[D_Y(G(x))^2]. \tag{3}$$

The photo discriminator uses the same form. These expressions average over examples and patch scores. Weighted cycle consistency is

$$\mathcal{L}_{\text{cyc}} = 10\big(\mathbb{E}_x\|F(G(x)) - x\|_1 + \mathbb{E}_y\|G(F(y)) - y\|_1\big), \tag{4}$$

where the implementation uses mean absolute pixel errors. Weighted identity losses are

$$\mathcal{L}_{\text{id}}^{G} = 5\mathbb{E}_y\|G(y) - y\|_1, \qquad \mathcal{L}_{\text{id}}^{F} = 5\mathbb{E}_x\|F(x) - x\|_1. \tag{5}$$

The logged generator totals are

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}}^{G} + \mathcal{L}_{\text{cyc}} + \mathcal{L}_{\text{id}}^{G}, \qquad \mathcal{L}_F = \mathcal{L}_{\text{adv}}^{F} + \mathcal{L}_{\text{cyc}} + \mathcal{L}_{\text{id}}^{F}.$$

Both totals include both cycle directions. Cycle and identity values reported later already include their loss weights.

## 4.2 Optimization and training configuration

**Table 3:** Configuration verified from notebook code and outputs.

| Setting | Value |
|---|---|
| Training framework | TensorFlow 2.20.0 / Keras |
| Resolution; batch size | $128 \times 128 \times 3$; 1 |
| Optimizers | Four independent Adam optimizers |
| Initial learning rate | $2 \times 10^{-4}$ |
| Adam parameters | $\beta_1 = 0.5$, $\beta_2 = 0.999$ |
| Cycle / identity weights | 10 / 5 |
| Replay pools | 50 images per domain |
| Completed epochs | 11 |
| batches per epoch | 20,655 |

Each step first updates both generators using TensorFlow gradient tapes. Generated images then pass through history pools before discriminator updates. Full pools randomly provide either a current image or a previously generated image. This reduces reliance on only the most recent outputs.

The learning-rate helper keeps the initial rate for five epochs, then linearly decays it toward zero for the selected training target. The rate is updated at step 1 and every 100 steps thereafter. Training calls target 2, 5, 8, and 12 epochs. Since the schedule depends on each call's target, this staged process is not one uninterrupted fixed schedule. The CSV does not preserve per-step learning rates.

Saved outputs show a Kaggle runtime with two visible Tesla T4 GPUs. No distributed-training speedup is claimed merely from device visibility. The first and last logged epochs took approximately 88.41 and 82.85 minutes respectively.

# 5 Experimental Results

## 5.1 Evaluation protocol

The notebook evaluates without updating weights. It exports real/generated images and computes FID using Clean-FID 0.1.35, `clean` mode, and Inception-v3 features. Each validation direction uses 1,000 real and 1,000 generated images; each test direction uses 679 of each. FID compares feature distributions; lower is better under a consistent protocol. It is **not an accuracy percentage** and does not independently establish identity preservation or correct colors.

**Table 4:** FID from saved evaluation JSON files.

| Split | Epoch | Photo → Sketch | Sketch → Photo |
|---|---|---|---|
| Validation | 5 | 119.9359 | 90.3537 |
| Validation | 11 | 135.8434 | 68.5298 |
| Test | 11 | 142.1067 | 76.5412 |

Sketch-to-photo validation FID improves by approximately 21.82 between epochs 5 and 11, while photo-to-sketch worsens by approximately 15.91. Additional training therefore did not improve both directions. Epoch 11 is the evaluated export, not a proven best checkpoint for both tasks. Test scores are higher than validation scores, with different split sizes also relevant to interpretation. Scores across the two directions involve different target distributions and should not be treated as a direct ranking of task difficulty.

## 5.2 Loss measurements

**Table 5:** Epoch-11 training means and full test means.

| Loss | Training | Test |
|---|---|---|
| $G$ total | 1.848827 | 1.887460 |
| $F$ total | 1.768538 | 2.002013 |
| $G$ adversarial | 0.997916 | 1.000715 |
| $F$ adversarial | 0.695187 | 0.877869 |
| Weighted cycle | 0.801366 | 0.837767 |
| $G$ weighted identity | 0.049544 | 0.048978 |
| $F$ weighted identity | 0.271989 | 0.286377 |
| Photo discriminator | 0.067536 | 0.080646 |
| Sketch discriminator | 0.000407 | 0.000005 |

The weighted training cycle loss decreases from 2.158613 at epoch 1 to 0.801366 at epoch 11. However, the very small sketch-discriminator loss and photo-to-sketch adversarial loss near one are consistent with a discriminator that still distinguishes generated sketches effectively. This suggests an adversarial imbalance, not perfect generation. Training discriminator losses use replay images, whereas test losses use current generated images.

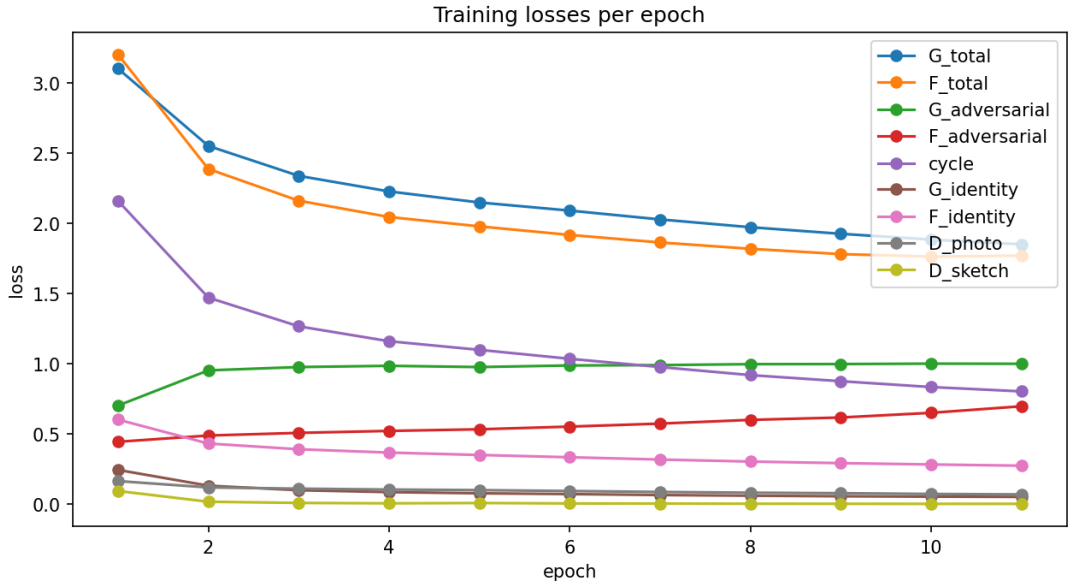


**Figure 3:** Training curves for generator, discriminator, and cycle losses.

**Figure 4:** Training versus test loss comparison at epoch 11.

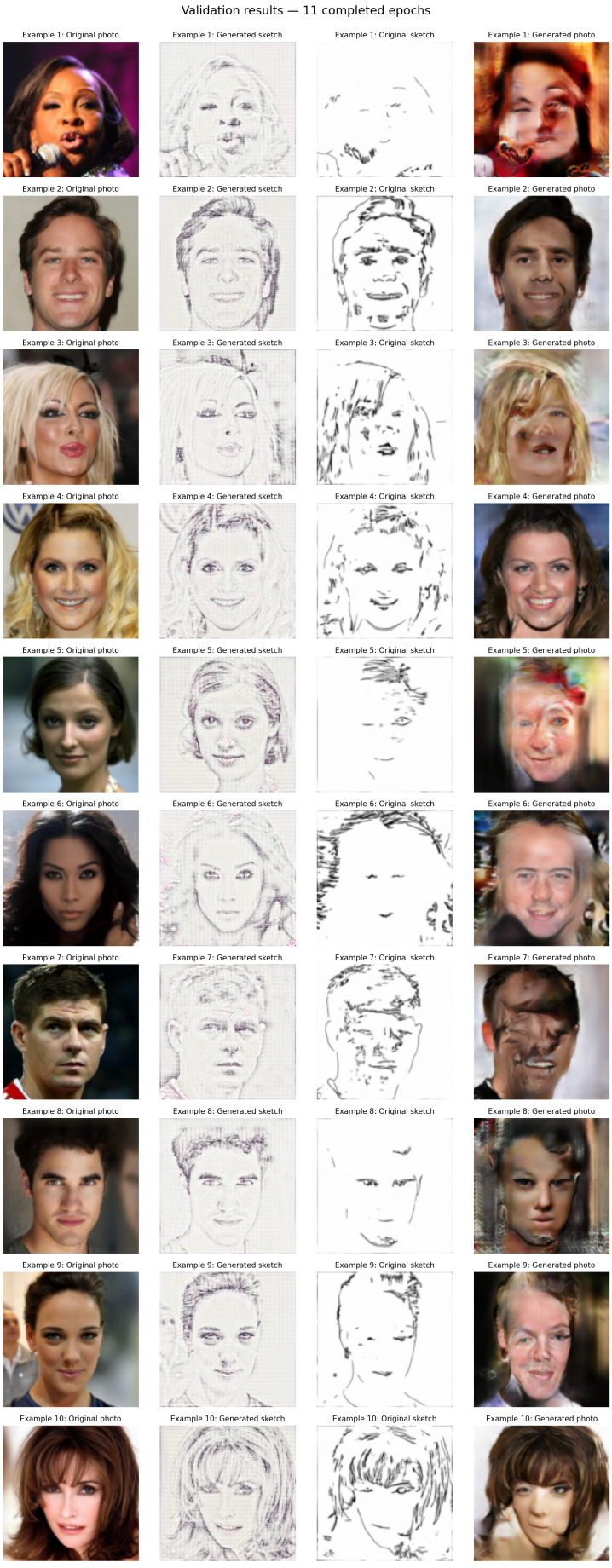


**Figure 5:** Validation inputs and generated outputs for both directions..

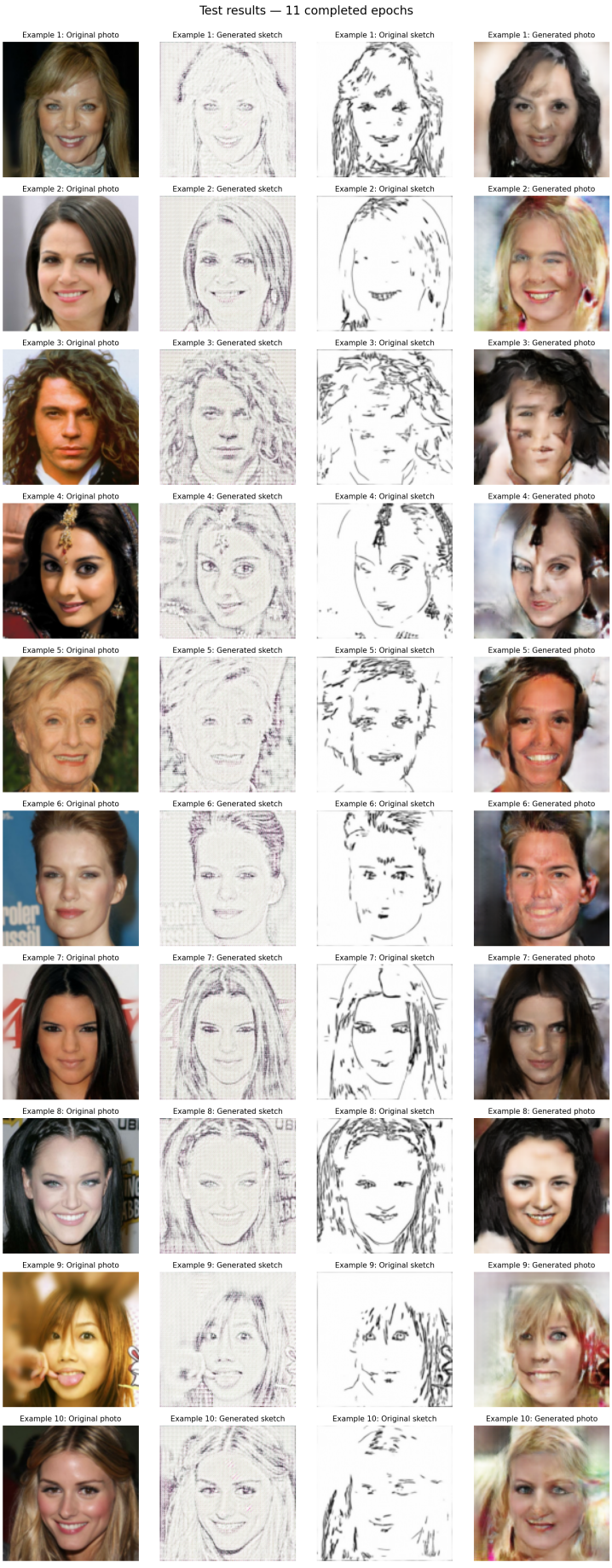


**Figure 6:** Held-out test photographs, sketches, and translated outputs.

**Figure 7:** Round-trip photo–sketch–photo and sketch–photo–sketch reconstructions.

## 6 Checkpointing and Model Export

Checkpoints contain both generators, both discriminators, four optimizer states, and the completed-epoch counter. The loop saves after every completed epoch and periodically within an epoch (default: every 1,000 steps). The manager retains the latest three checkpoints, rather than permanently retaining every epoch's checkpoint.

Restoration selects the latest checkpoint, with a numeric-index fallback for stale paths. The notebook restores `ckpt-254` and reports 11 completed epochs: 254 is a save counter, not an epoch number. Data iterator position, image pools, and full random state are not restored, so resumption preserves learning progress but not exact replay of the original step sequence.

Export produces both `.weights.h5` and `.keras` generator files. Flask builds the architecture and loads weights once at startup, with full-model files as a fallback. It runs shape/range self-checks before serving requests. Local checks used the existing Python 3.14 / TensorFlow 2.22.0-rc0 environment; requirements retain TensorFlow 2.20.0 to match training.

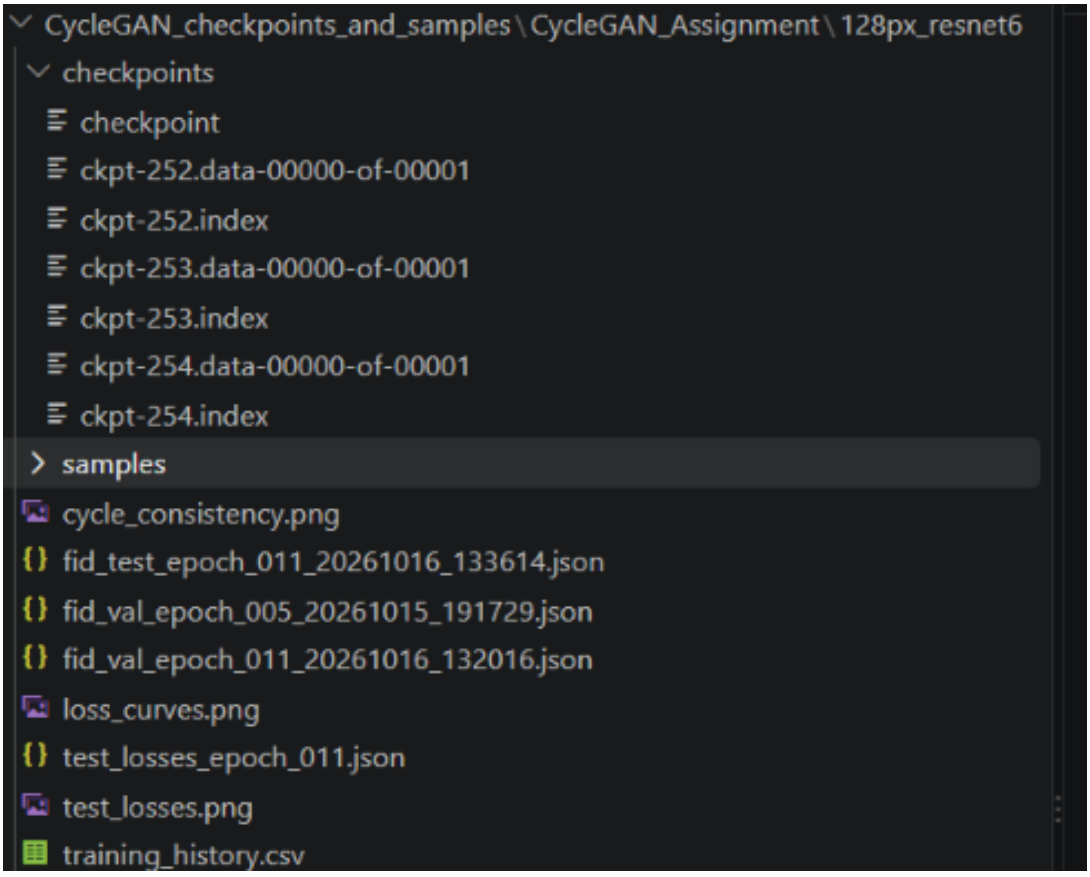


**Figure 8:** Snapshot of all the files checkpoint files.

# 7 Flask Backend and React Interface

## 7.1 Application workflow

The application in `cyclegan_ui/` uses React/TypeScript and Vite with a Flask backend. Users upload an image, drag and drop a file, or capture a camera frame. Modes are **Auto detect**, **Picture to Sketch**, and **Sketch to Picture**. Manual selection overrides automatic routing. The original image and output appear side by side, with a separate model-input preview, inference duration, PNG download, and reset control.

Flask serves the built React app locally. The endpoints are:

- `GET /`: application page; `GET /health`: model readiness.
- `POST /api/preview`: detect and preprocess without translation.
- `POST /api/convert`: preprocess, infer, and return images/metadata.

Requests accept multipart uploads or base64 captures. Supported image formats are PNG, JPEG, BMP, and WebP, with a 10 MB decoded-image limit. Validation covers malformed/missing inputs, invalid options, and oversized files. Models are cached at startup, and calls use a threading lock and `training=False`. No external inference API or runtime CDN is required. Camera support uses captured frames, not continuous video translation; no guaranteed real-time frame rate is claimed.

## 7.2 Automatic input detection

For original RGB pixels scaled to $[0, 1]$, the heuristic computes

$$s = \text{mean}(\max(R, G, B) - \min(R, G, B)), \tag{6}$$

$$w = \text{mean}\left[(R + G + B)/3 > 0.85\right]. \tag{7}$$

An image is classified as a sketch if $s < 0.05$ and $w > 0.50$, otherwise as a photo. Detection precedes cropping/cleanup. Confidence is an uncalibrated score based on threshold distances, not measured dataset accuracy. Grayscale photos and dark sketches can confuse it; manual routing remains available.

The detector-evaluation script supports validation/test evaluation and validation threshold search. Full detector accuracy has not been measured locally because the dataset is absent. No 98% accuracy claim or trained logistic-regression detector is made. Thresholds remain 0.05 and 0.50.

## 7.3 Application preprocessing and optional enhancement

The app fixes EXIF orientation, composites transparent images onto white, converts to RGB, and downsizes very large inputs. For photos, optional OpenCV Haar-cascade detection selects the largest face and expands its crop to approximately 1.8 times the detected size. Failure falls back to a center square with a warning. Disabling photo crop keeps the whole image. Sketches skip face detection and use a center square.

**Clean up sketch** is optional, off by default, and used only for sketch-to-photo input. It converts to grayscale, applies a bilateral filter, stretches intensities between the 2nd and 98th percentiles when their range is sufficient, and blends 75% processed with 25% original grayscale. The result is replicated into three channels. These are conventional OpenCV operations, not

another generative model. Improvement in generated-photo quality has not been established experimentally.

Final resizing and normalization match the notebook. Display pixels are recovered as

$$I_{\text{out}} = \text{uint8}\left(\text{round}(\text{clip}((y+1)127.5, 0, 255))\right).$$

Generated sketches optionally receive grayscale conversion and contrast enhancement; generated photos do not. Results are enlarged to $512 \times 512$ for display/download, but native generation remains $128 \times 128$.

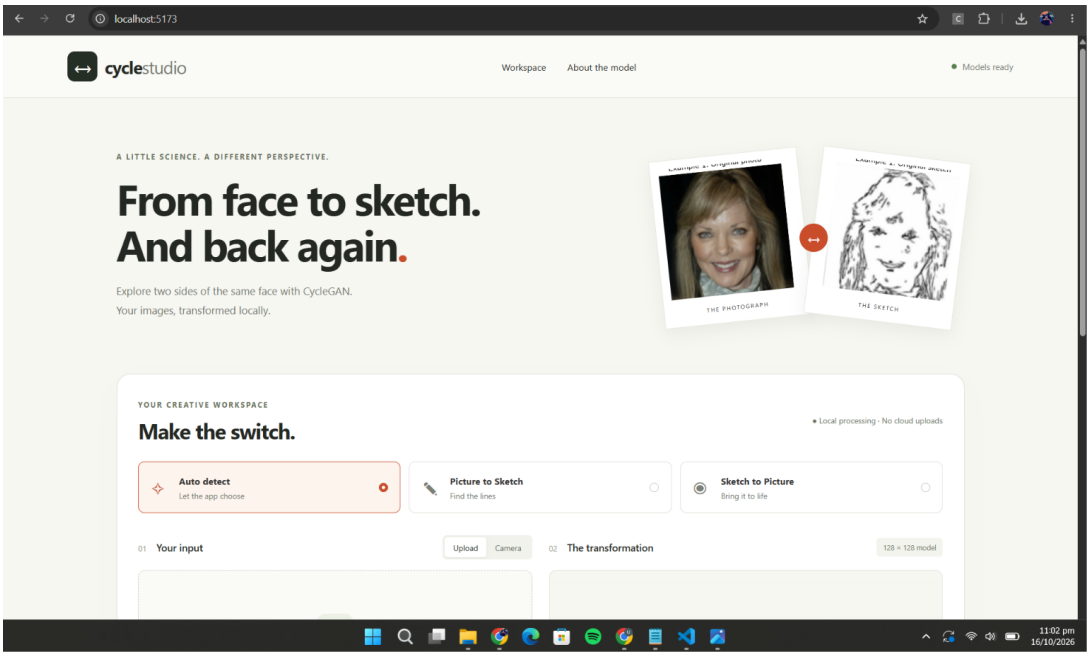


**Figure 9:** React workspace showing mode selection, input, generated result, and download controls.

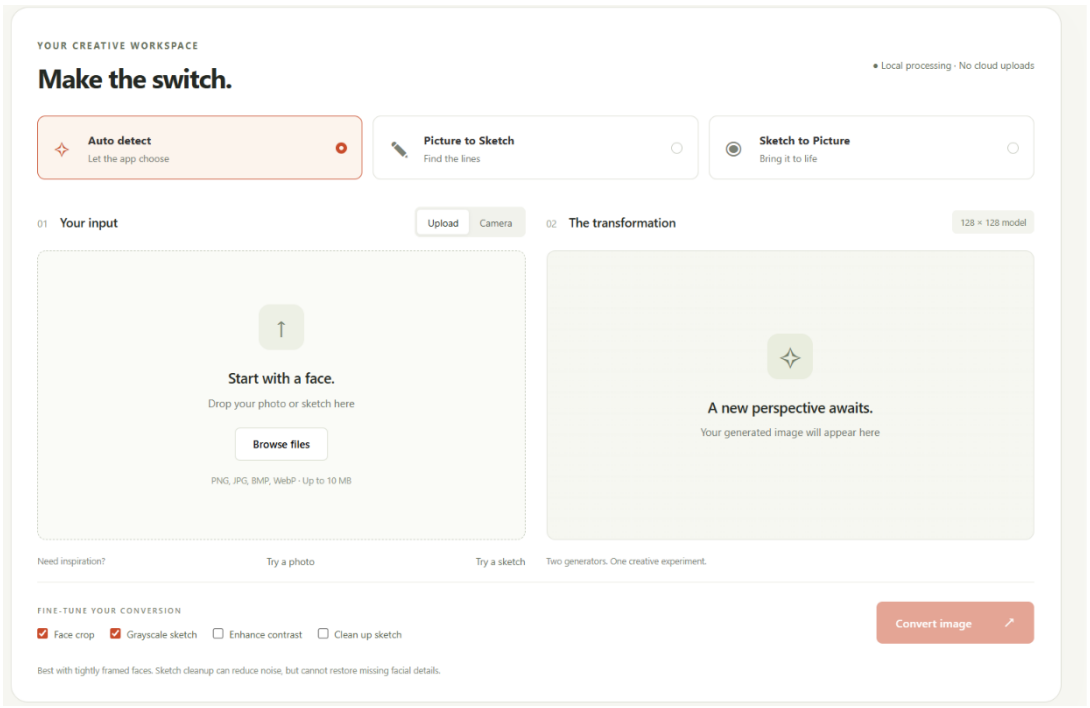


**Figure 10:** React workspace showing mode selection, input, generated result, and download controls.

## 7.4 Sketch-to-Photo Translation Demo

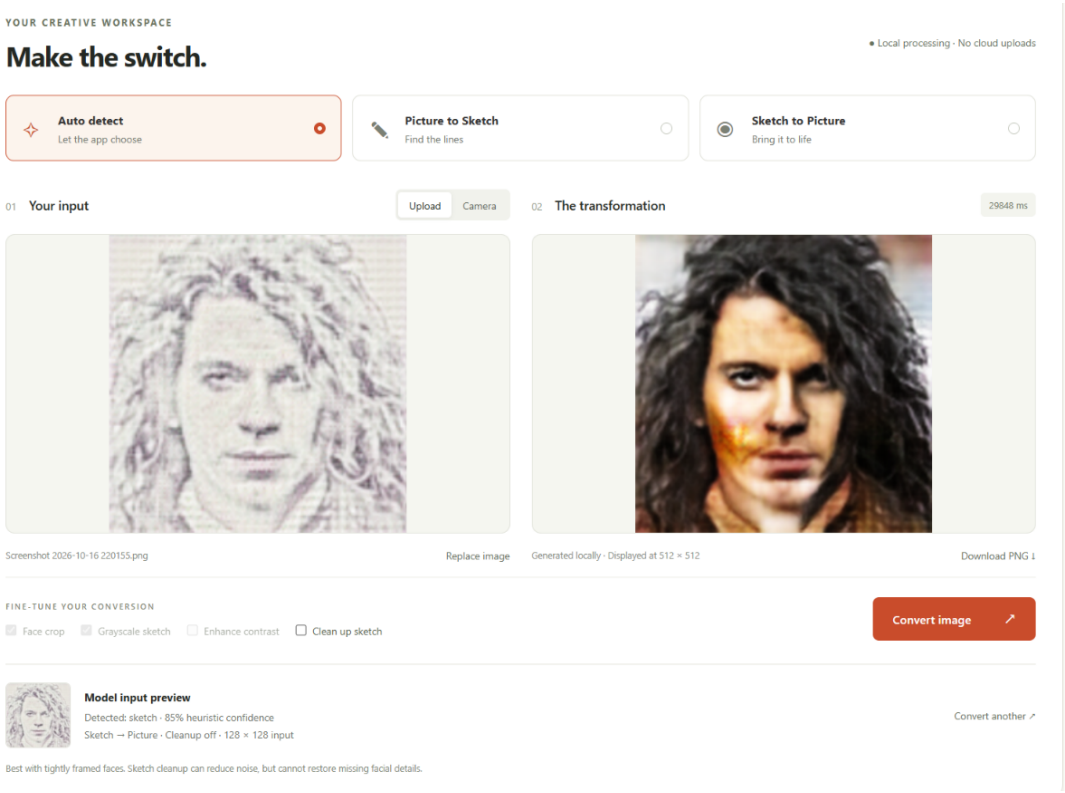


**Figure 11:** Sample Sketch 1

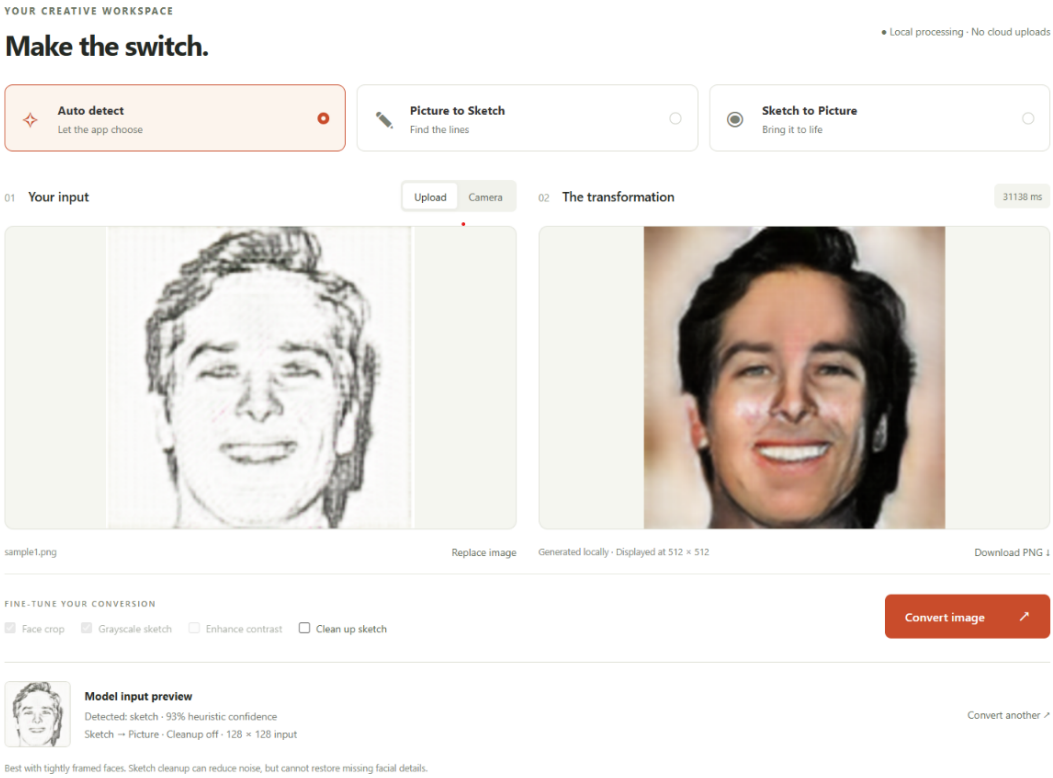


**Figure 12:** Sample Sketch 1

## 7.5 Photo-to-Sketch Translation Demo

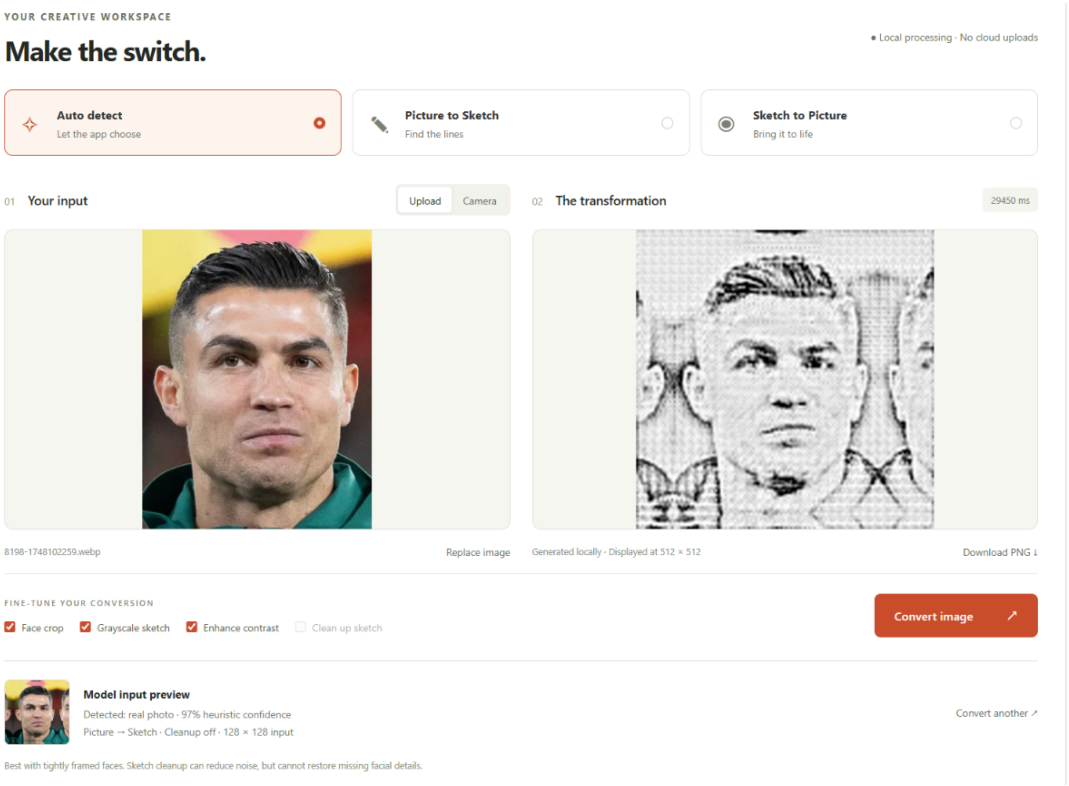


**Figure 13:** Sample Photo 1

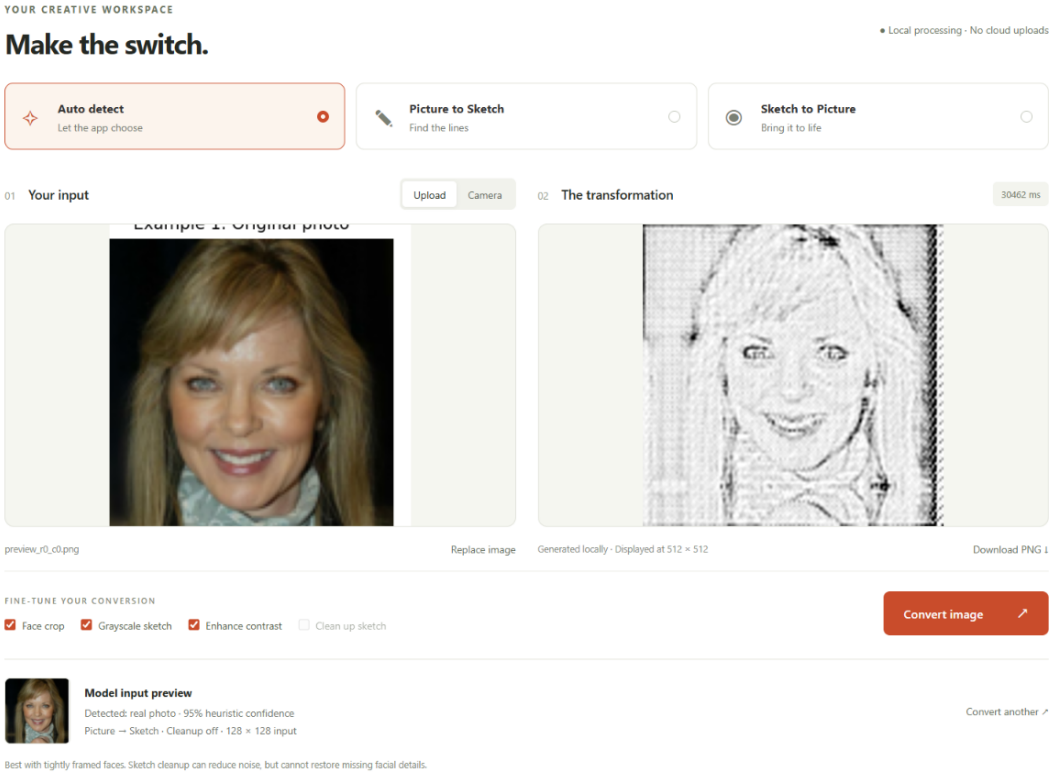


**Figure 14:** Sample Photo 2

# 8 Verification and Limitations

The React TypeScript production build passed. Fourteen automated checks passed across the main and follow-up runs, covering frontend/health responses, both real-model directions, PNG/grayscale output, synthetic detector cases, cleanup routing, unchanged detection before cleanup, invalid/oversized inputs, exact TensorFlow resize/normalization, multipart input, and corrupt-file handling. Float32 boundary rounding is allowed in input-range checks.

These checks establish software behavior, not perceptual quality or detector accuracy. Physical webcam access and complete browser interaction testing remain manual checks. Figure placeholders must be replaced with actual evidence before submission.

Key limitations are:

- Eleven completed epochs and $128 \times 128$ generation limit training and detail; enlargement does not establish high-resolution generation.
- Mixed FID trends show that falling cycle loss alone does not establish realistic or identity-preserving outputs.
- Sparse sketches can generate distorted faces; cleanup cannot reliably restore missing facial structure.
- Original colors are not uniquely recoverable from grayscale sketches. The generator predicts colors from learned patterns and receives no cached original photograph.
- Automatic detection is heuristic and uncalibrated; its dataset accuracy remains unmeasured.
- App cropping, cleanup, and sketch postprocessing differ from notebook evaluation. Saved FID values are not measurements of the postprocessed web app.
- Resumption does not restore exact data order, replay pools, or full random state; staged learning-rate targets complicate exact reproducibility.

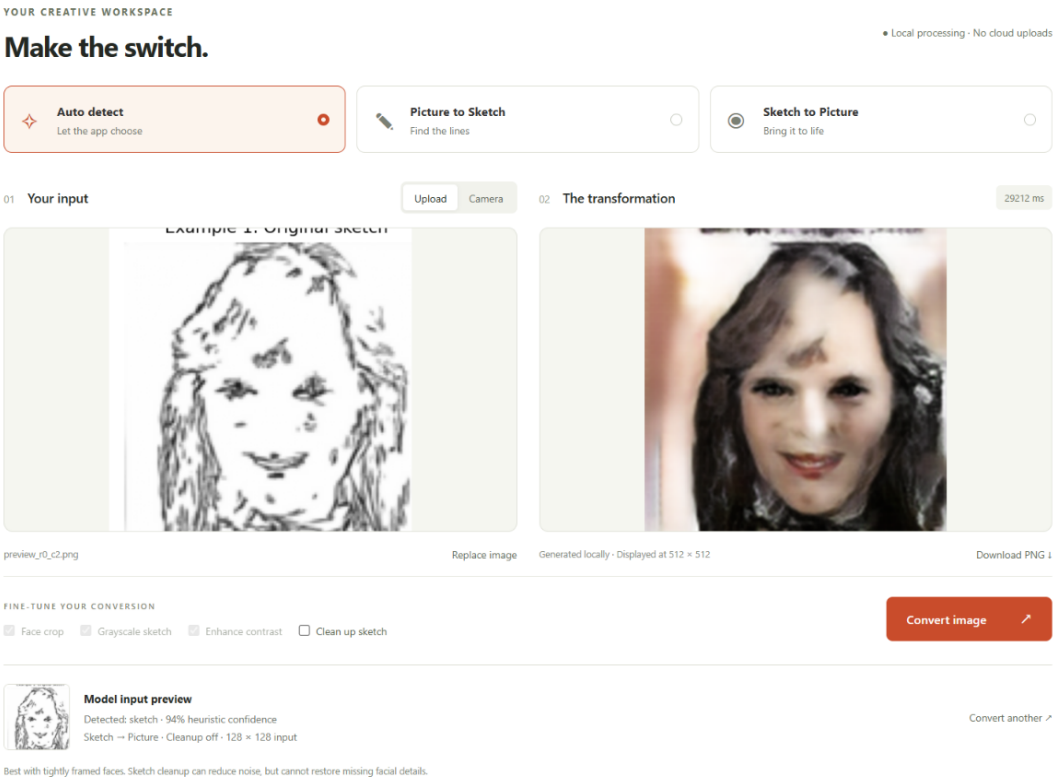


**Figure 15:** Distorted face from bad sketch example

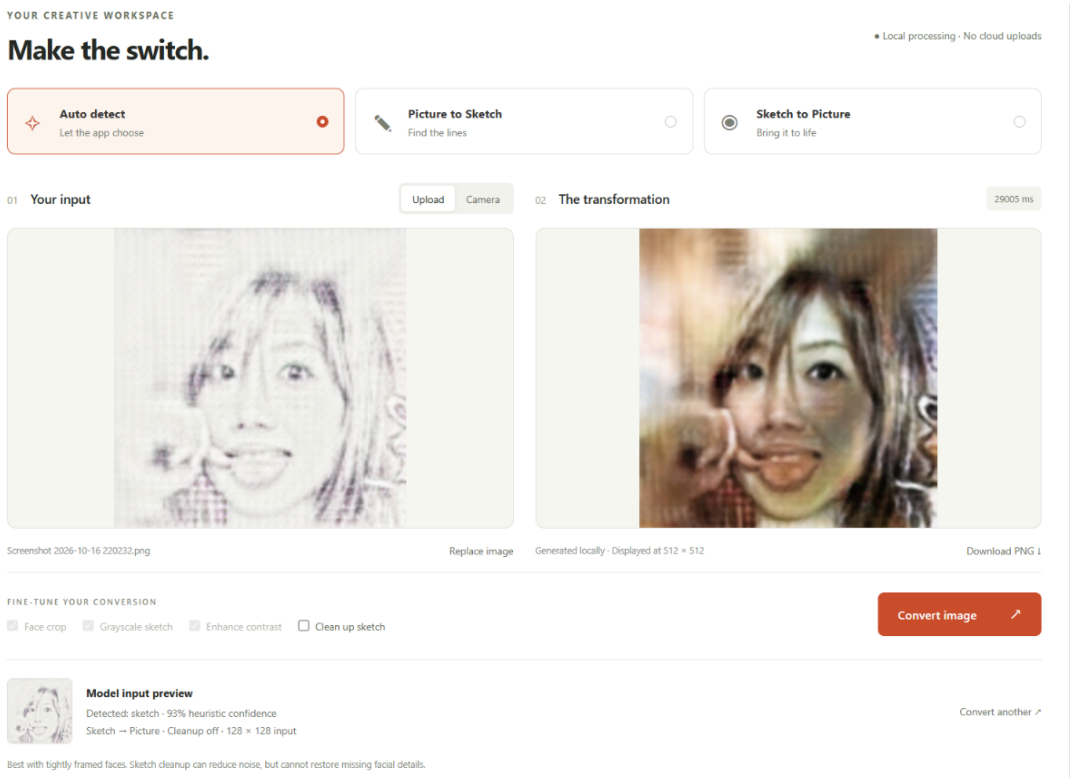


**Figure 16:** Distorted face from bad sketch example 2

Future work should evaluate checkpoint selection separately for both directions, inspect discriminator balance, record a fixed learning-rate schedule, and compare cleanup on/off on identical

validation sketches. Detector improvements require train/validation data and a held-out test evaluation. Higher-resolution and longer training should be assessed as new experiments rather than assumed improvements.

## 9 Conclusion

The project combines end-to-end CycleGAN training, checkpoint recovery, quantitative evaluation, and a local React/Flask demonstration. At the final recorded epoch 11, test FID is 142.1067 for photo-to-sketch and 76.5412 for sketch-to-photo. The application supports upload/camera input, automatic/manual routing, optional cleanup, and image download. The results establish a working baseline while identifying translation-quality and detector-evaluation limitations.

## A Artifacts and Run Commands

| Artifact | Purpose |
|---|---|
| `cyclegan-q2-notebook.ipynb` | Training, restoration, evaluation, and export. |
| `models_and_weights/` | Exported generators and weights. |
| `cyclegan_ui/` | Backend, React frontend, weights, tests, and README. |
| `CycleGAN_checkpoints_and_samples/` | Checkpoints, samples, history, and evaluation JSON files. |

From the Q2 directory in PowerShell:

```
.\venv\Scripts\python.exe cyclegan_ui\app.py
# Open http://127.0.0.1:5000 after startup checks.
npm.cmd --prefix cyclegan_ui/frontend run build
# In another terminal, with Flask running:
.\venv\Scripts\python.exe -m pytest cyclegan_ui\tests -q
```

Fresh installation and detector-evaluation instructions are in `cyclegan_ui/README.md`.

## References

[1] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*. ICCV, 2017. https://arxiv.org/abs/1703.10593.

[2] Almightyj. *Person Face Sketches*. Kaggle dataset. https://www.kaggle.com/datasets/almightyj/person-face-sketches.

[3] Project notebook and saved run artifacts: `cyclegan-q2-notebook.ipynb`, `training_history.csv`, epoch-5/epoch-11 validation FID JSON files, `fid_test_epoch_011_20261016_133614.json`, and `test_losses_epoch_011.json`.